

Spell Indicator

Overview

The Spell Indicator System is a comprehensive Unity package designed for creating highly customizable spell and range indicators in games. Built specifically for Unity's Universal Render Pipeline (URP), this system provides developers with powerful tools to create visually stunning and performant spell casting indicators, area-of-effect markers, and interactive range displays.

Key Features

Versatile Shape Support

- **Circle & Circle Cone:**
 - Perfect for radial spells, area-of-effect abilities, and circular range displays
 - Circle Cone: Combine circular range with angular restrictions for sector-based abilities
- **Rectangle & Rectangle Line**
 - Ideal for directional spells and rectangular zones
 - Nine-Slice: Scalable rectangle indicators with customizable borders

Advanced Visual Customization

- **Blend Mode Flexibility**
 - Configurable blend modes for both source and destination
 - Support for additive and alpha blending
- **Texture & Color Systems**
 - Base Layer: Primary indicator appearance with texture support
 - Fill Layer: Secondary overlay for progress, cooldown, or charge effects
 - Boundary Lines: Optional edge highlighting for precise area definition
 - Rotation Support: Animated texture rotation with customizable speed
- **Dynamic Progress Indicators**
 - Real-time progress visualization for charging spells
 - Angular fill options for directional abilities
 - Configurable fill patterns and falloff effects

Performance Optimizations

- GPU-based rendering using shaders
- Optimized geometry with minimal vertex count
- Zero CPU overhead for animation updates
- Compatible with Unity's SRP Batcher
- Runtime Parameter Adjustment without material recreation
- Mobile-Optimized variants for performance scaling

Usage Guide

Quick Start

- **Create Indicators:** Use the provided materials or create new ones using the included shaders
- **Configure Settings:** Adjust parameters in the material inspector for your desired appearance
- **Runtime Control:** Modify material properties via script for dynamic effects

Material Configuration

- **Circle Shader Properties**
 - **Main Settings**
 - ◆ **Radius:** Controls the overall size of the circular indicator (0-20 units)

- ◆ **Alpha:** Master transparency control (0-1)
- ◆ **Alpha Falloff:** Transparency falloff (0-1)
- **Base Layer**
 - **Texture:** Primary texture for the indicator surface
 - **Color:** Base tint color
 - **Intensity:** Emission multiplier for glow effects
 - **Radius:** Inner radius scaling (0-1)
 - **Angle:** Angular coverage for sector displays (0-1 = 0-360°)
 - **Rotation Speed:** Automatic texture rotation speed
- **Fill Layer (Optional)**
 - **Enable:** Toggle secondary fill overlay
 - **Fill Angle:** Angular fill range for directional indicators
 - **Alpha Falloff:** Fill layer transparency control
- **Boundary Lines (Optional)**
 - **Enable:** Toggle edge highlighting
 - **Color:** Boundary line color
 - **Thickness:** Line thickness (0-0.1)
 - **Intensity:** Emission strength for boundary glow
- **Rectangle Shader Properties**
 - **Main Settings**
 - ◆ **Width:** Horizontal size in world units (0.1-20)
 - ◆ **Height:** Vertical size in world units (0.1-20)
 - ◆ **Alpha:** Master transparency control
 - ◆ **Alpha Falloff:** Transparency falloff (0-1)
 - **Nine-Slice Support**
 - ◆ **Enable:** Toggle nine-slice scaling
 - ◆ **Border Sizes:** Individual control for top, bottom, left, right borders
 - ◆ **UV Mapping:** Precise texture coordinate control for scalable elements

Script Integration

- **Runtime Parameter Updates**

```
// Update spell radius
spellMaterial.SetFloat("_MainRadius", 5.0f);

// Change fill progress for charging effect
spellMaterial.SetFloat("_FillProgress", chargeAmount);

// Modify colors dynamically
spellMaterial.SetColor("_BaseColor", Color.red);
spellMaterial.SetColor("_FillColor", Color.yellow);
```

- **Performance Tips**
 - Cache material property IDs for optimal performance
 - Use MaterialPropertyBlocks for per-instance variations
 - Leverage Unity's SRP Batcher compatibility for draw call optimization

Performance Characteristics

- **Vertex Count:** Optimized geometry (4-8 vertices per indicator)
- **Shader Complexity:** Minimal ALU operations with variants
- **Batching:** Full SRP Batcher compatibility

Best Practices

- **LOD System:** Implement distance-based quality scaling
- **Pooling:** Use object pooling for frequently created/destroyed indicators

Enjoying this package?

- Give it a quick rating or leave a short review on the Unity Asset Store.
- Your feedback helps us improve and add even more features!
- [Review on Unity Asset Store](#)